Estacionamientos	Desarrollo	Crear las Vistas *SpaceViewList* y *SpaceViewDetail*. (*Django*)	1	Patricio Cruz	En Proceso
	Desarrollo	Crear la página *ParkingShow.jsx* y componente *ParkingCard.jsx*. (*React, HTML5, JavaScript*)	1	David Carrascal	En Proceso
	Desarrollo	Estilizar el *ParkingShow.jsx* y *ParkingCard.jsx*. (*MUI, CSS*)	2	Patricio Cruz	En Proceso
	Pruebas	Realizar las pruebas sobre la visualización de estacionamientos.	0.5	David Carrascal	En Proceso

4.3.3.2. Sprint 1 - Reuniones diarias (*Daily Scrum*)

Por medio de la planeación realizada en el *sprint* 1, se llevaron a cabo reuniones diarias, cuya duración máxima fue de 15 minutos por día, en el que se registraron las actividades realizadas, los obstáculos encontrados para la culminación de las tareas y la definición de próximas tareas. Además, se empleó la aplicación *Jira* para dar seguimiento de las distintas tareas de ingeniería que están asociadas a las correspondientes historias de usuario, incluyendo sus estimaciones.

4.3.3.2.1. Historia de Usuario 1: Inicio de Sesión (Login)

La información detallada en la historia se puede hallar en el anexo V. Para el desarrollo de la historia 1, se utilizó un *template* gratuito de *MUI* con licencia del *MIT*, esta plantilla únicamente incorpora la funcionalidad de inicio de sesión, por lo cual se creó las páginas *AuthLogin.jsx* y *Login3.jsx,* para que se conecte con el *backend* y utilice el correo electrónico como parte de las credenciales para iniciar sesión, como se aprecia en la figura 8 vista desde la computadora y la figura 9 vista desde el teléfono.

Figura 8. Interfaz de Login3.jsx, vista de escritorio

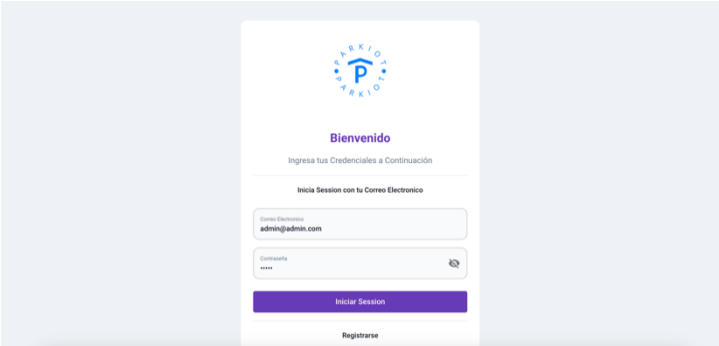
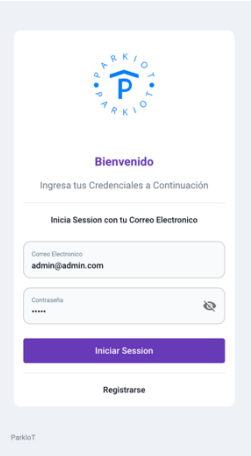


Figura 9. Interfaz de Login3.jsx, vista móvil



La pantalla para iniciar sesión fue realizada utilizando componentes de *MUI* y la biblioteca *React*, como se visualiza en la figura 10**.** Además, se empleó la librería de *Formik* para manipular los formularios, validarlos y manejar los mensajes de error, y para conectar con el *backend* se utilizó la librería de *Axios*, estas aplicaciones se indican en la figura 11.

Figura 10. Código de Login3.jsx

```
import { Link } from 'react-router-dom';

// material-ui
import { useTheme } from '@mui/material/styles';
import { Divider, Grid, Stack, Typography, useMediaQuery } from '@mui/material';

// project imports
import AuthWrapper1 from '../AuthWrapper1';
import AuthCardWrapper from '../AuthCardWrapper';
import AuthLogin from '../auth-forms/AuthLogin';
import Logo from 'ui-component/Logo';
import AuthFooter from 'ui-component/cards/AuthFooter';

// assets

// ==============================|| AUTH3 - LOGIN ||============================== //

const Login = () => {
  const theme = useTheme();
  const matchDownSM = useMediaQuery(theme.breakpoints.down('md'));

  return (
    <AuthWrapper1>
      <Grid container direction="column" justifyContent="flex-end" sx={{ minHeight: '100vh' }}>
        <Grid item xs={12}>
          <Grid container justifyContent="center" alignItems="center" sx={{ minHeight: 'calc(100vh - 68px)' }}>
            <Grid item sx={{ m: { xs: 1, sm: 3 }, mb: 0 }}>
              <AuthCardWrapper>
                <Grid container spacing={2} alignItems="center" justifyContent="center">
                  <Grid item sx={{ mb: 3 }}>
                    <Link to="#">
                      <Logo width={150}/>
                    </Link>
```

Figura 11. Código de AuthLogin.jsx

```
          <Typography variant="subtitle1">Inicia Session con tu Correo Electronico</Typography>
        </Box>
      </Grid>
    </Grid>

    <Formik
      initialValues={{
        email: 'admin@admin.com',
        password: 'admin',
        submit: null
      }}
      validationSchema={Yup.object().shape({
        email: Yup.string().email('El correo debe ser valido').max(255).required('Correo requerido'),
        password: Yup.string().max(255).required('Contraseña requerida')
      })}
      onSubmit={async (values, { setErrors, setStatus, setSubmitting }) => {
        try {
          axios
            .post(
              `${configData.API_SERVER}/api/users/login`,
              {
                password: values.password,
                email: values.email
              },
              {
                headers: {
                  'content-type': 'application/json'
                }
              }
            )
            .then((response) => {
              if (response.data?.success) {
                                    cher({
                                    ACCOUNT_INITIALIZE,
```

En el *backend* se utilizó *Django*, con un servidor previamente configurado para que utilice autenticación con *JSON Web Tokens*, este servidor tiene licencia *MIT* para el uso libre, a partir de esa base se creó la vista *LoginViewSet,* que se muestra en la figura 12 y se modificó el modelo *User*, para que se adapte a las necesidades de la aplicación, como se muestra en la figura 13**.**

Figura 12. Código de la Vista LoginViewSet

```
from rest_framework import viewsets, mixins
from rest_framework.response import Response
from rest_framework import status
from rest_framework.permissions import AllowAny

from api.authentication.serializers import LoginSerializer


class LoginViewSet(viewsets.GenericViewSet, mixins.CreateModelMixin):
    permission_classes = (AllowAny,)
    serializer_class = LoginSerializer

    def create(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        return Response(serializer.validated_data, status=status.HTTP_200_OK)
```

Figura 13. Código del Modelo User

```
class UserManager(BaseUserManager):
    def create_user(self, username, email, password=None, first_name="", last_name="", **kwargs):
        """Create and return a `User` with an email, username and password."""
        if username is None:
            raise TypeError("Users must have a username.")
        if email is None:
            raise TypeError("Users must have an email.")

        # Identifieds like qrcode in base64 and uuid pure
        ids = create_qrcode(username, URL_FRONTEND)

        user = self.model(username=username, email=self.normalize_email(email),
                          first_name=first_name, last_name=last_name,
                          qrcode=ids[0], uuid=ids[1])
        user.set_password(password)
        user.save(using=self._db)

        return user

    def create_superuser(self, username, email, password):
        """
        Create and return a `User` with superuser (admin) permissions.
        """
        if password is None:
            raise TypeError("Superusers must have a password.")
        if email is None:
            raise TypeError("Superusers must have an email.")
        if username is None:
            raise TypeError("Superusers must have an username.")

        user = self.create_user(username, email, password)
        user.is_superuser = True
```

4.3.3.2.2. Historia de Usuario 2: Registro de Clientes

La información detallada en la historia se puede hallar en el anexo V. Para el desarrollo de la historia 2, se siguió utilizando el modelo para el registro de usuarios que tiene el *template* de *MUI*, pero se adaptó a las necesidades de la aplicación, creando así las páginas *AuthRegister.jsx* y *Register3.jsx*, para que se conecten al *backend* y realicen el proceso de registro con las funciones creadas, a la página de registro se le agregó los campos de nombres y cédula como se muestra en la figura 14, vista desde la computadora y la figura 15 vista desde el teléfono.

Figura 14. Interfaz de Register3.jsx, vista de escritorio

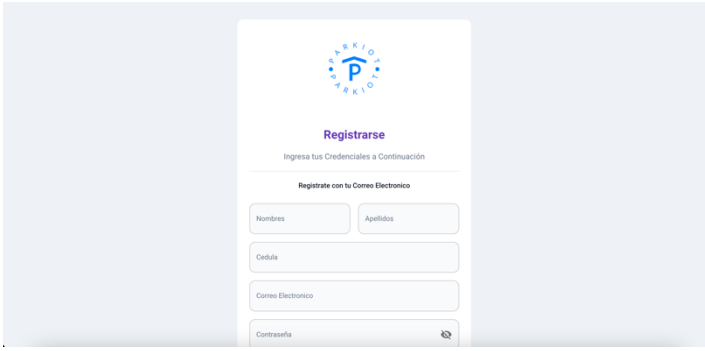


Figura 15. Interfaz de Register3.jsx, vista móvil

Se utilizó librerías de *React* y componentes de *MUI* para crear la página *Register3.jsx,* como se muestra en la figura 16, dentro de esta página se contiene *AuthRegister.jsx* que implementa la librería de *Formik* para manejar formulario, validarlos y manejar errores, además, *Axios* sirve para conectar con el *backend*, como se visualiza en la figura 17.

Figura 16. Código de Register3.jsx

```
import { Link } from 'react-router-dom';

// material-ui
import { useTheme } from '@mui/material/styles';
import { Divider, Grid, Stack, Typography, useMediaQuery } from '@mui/material';

// project imports
import AuthWrapper1 from '../AuthWrapper1';
import AuthCardWrapper from '../AuthCardWrapper';
import Logo from 'ui-component/Logo';
import AuthRegister from '../auth-forms/AuthRegister';
import AuthFooter from 'ui-component/cards/AuthFooter';

// assets

// ==============================|| AUTH3 - REGISTER ||============================== //

const Register = () => {
  const theme = useTheme();
  const matchDownSM = useMediaQuery(theme.breakpoints.down('md'));

  return (
    <AuthWrapper1>
      <Grid container direction="column" justifyContent="flex-end" sx={{ minHeight: '100vh' }}>
        <Grid item xs={12}>
          <Grid container justifyContent="center" alignItems="center" sx={{ minHeight: 'calc(100vh - 68px)' }}>
            <Grid item sx={{ m: { xs: 1, sm: 3 }, mb: 0 }}>
              <AuthCardWrapper>
                <Grid container spacing={2} alignItems="center" justifyContent="center">
                  <Grid item sx={{ mb: 3 }}>
                    <Link to="#">
                      <Logo width={150}/>
                    </Link>
```

Figura 17. Código de AuthRegister.jsx

```
    </Grid>

    <Formik
      initialValues={{
        first_name: '',
        last_name: '',
        email: '',
        password: '',
        submit: null
      }}
      validationSchema={Yup.object().shape({
        email: Yup.string().email('Must be a valid email').max(255).required('El correo es requerido'),
        password: Yup.string().max(255).required('La contrasena es requerida')
      })}
      onSubmit={async (values, { setErrors, setStatus, setSubmitting }) => {
        if (checked) {
          try {
            axios
              .post(`${configData.API_SERVER}/api/users/register`, {
                first_name: values.first_name,
                last_name: values.last_name,
                email: values.email,
                password: values.password,
                username: extraerUsuarioCorreo(values.email)
              })
              .then((response) => {
                if (response.data.success) {
                  if (scriptedRef.current) {
                    setStatus({ success: true });
                    setSubmitting(false);
                  }
                  navigate('/login');
                } else {
```

En el *backend* se creó la vista *RegisterViewSet*, que mediante la función *create*, permite crear instancias del objeto *User,* que a su vez se guarda en la base de datos *MySQL* mediante el *ORM* de *Django*, la lógica se puede apreciar en el código de la figura 18. Dentro del proceso de creación del usuario, se utilizó el serializador *RegisterSerializer* para validar los campos correspondientes al modelo *User* como se visualiza en la figura 19.

Figura 18. Código de la vista RegisterViewSet

```
from rest_framework import viewsets, status
from rest_framework.response import Response
from rest_framework.permissions import AllowAny

from api.authentication.serializers import RegisterSerializer

Patricio Cruz, 3 weeks ago | 1 author (Patricio Cruz)
class RegisterViewSet(viewsets.ModelViewSet):
    http_method_names = ["post"]
    permission_classes = (AllowAny,)
    serializer_class = RegisterSerializer

    def create(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)      Patricio Cruz, 3 weeks ago • Actualizacion Completa

        serializer.is_valid(raise_exception=True)
        user = serializer.save()

        return Response(
            {
                "success": True,
                "userID": user.id,
                "msg": "The user was successfully registered",
            },
            status=status.HTTP_201_CREATED,
        )
```

Figura 19. Código del Serializador RegisterSerializer

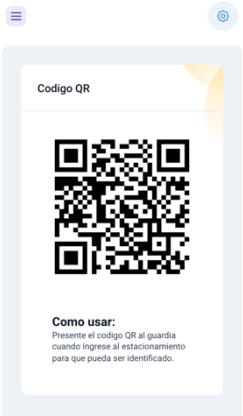
```
from rest_framework import serializers
from rest_framework.exceptions import ValidationError
from django.core.exceptions import ObjectDoesNotExist
from api.user.models import User


Patricio Cruz, 3 weeks ago | 1 author (Patricio Cruz)
class RegisterSerializer(serializers.ModelSerializer):
    password = serializers.CharField(min_length=4, max_length=128, write_only=True)
    username = serializers.CharField(max_length=255, required=True)
    email = serializers.EmailField(required=True)

    Patricio Cruz, 3 weeks ago | 1 author (Patricio Cruz)
    class Meta:
        model = User
        fields = ["id", "username", "password", "email", "is_active", "date", "first_name", "last_name"]

    def validate_username(self, value):
        try:
            User.objects.get(username=value)
        except ObjectDoesNotExist:
            return value
        raise ValidationError({"success": False, "msg": "Nombre de Usuario ya Existente."})

    def validate_email(self, value):
        try:
            User.objects.get(email=value)
        except ObjectDoesNotExist:
            return value
        raise ValidationError({"success": False, "msg": "Correo Electronico ya Existente"})

    def create(self, validated_data):
        return User.objects.create_user(**validated_data)      Patricio Cruz, 3 weeks ago • Actualizacion Completa
```

4.3.3.2.1. Historia de Usuario 3: Identificador QR

La información detallada en la historia se puede hallar en el anexo V. Para el desarrollo de la interfaz del Identificador *QR* se creó la página *QrcodeShow.jsx*, donde se decidió solo mostrar el código *QR* de manera que sea lo único visible para la interacción con el guardia al momento de escanear el código, debajo se definió una pequeña guía para que el usuario conozca la utilidad que tiene el código *QR*, como se visualiza en la figura 20 vista desde el computador y la figura 21 vista desde el teléfono.

Figura 20. Interfaz de QrcodeShow.jsx, vista de escritorio

Figura 21. Interfaz de QrcodeShow.jsx, vista móvil



Se utilizó las librerías de *React* y *MUI* para crear el componente *QrcodeCard.jsx* que presenta el código *QR*, ya que este código como imagen está comprimido en *base64* para que pueda ser almacenado, este es presentado con el componente *Box* que simula la etiqueta de imagen *HTML* convencional, como se muestra en la figura 22. Además, se creó el componente *QrcodeShow.jsx* que utiliza la librería de *Axios* para traer la información del *backend*, como se visualiza en la figura 23.

Figura 22. Código de QrcodeCard.jsx

```
import React from 'react';
import { Box } from '@mui/material';
const QrcodeCard = ({ qrcode }) => {
  return (
    <>
      <Box
        component="img"
        sx={{
          height: {
            xs: 280,
            md: 400,
            xl: 450
          },
          width: {
            xs: 280,
            md: 400,
            xl: 450
          }
        }}
        src={`${qrcode}`}      Patricio Cruz, 3 weeks ago • Actualizacion Completa
        alt='Codigo QR con Informacion del Usuario'
      />
    </>
  );
};

export default QrcodeCard;
```

Figura 23. Código de QrcodeShow.jsx

```
import { useEffect, useState } from 'react';
import { useSelector } from 'react-redux';
import axios from 'axios';
// import MainCard from 'ui-component/cards/MainCard';
import { Box, Typography } from '@mui/material';
// import { Typography } from '@mui/material'

import QrcodeCard from 'components/QrcodeCard';
import configData from '../../config';
import GeneralBack from 'components/GeneralBack';

const QrcodeShow = () => {
  const account = useSelector((state) => state.account);
  const [qrcode, setQrcode] = useState(null);

  const callApi = async () => {
    axios
      .get(`${configData.API_SERVER}/api/users/qrcode/${account.user?._id}/` , {
        headers: {
          Authorization: `${account.token}`
        }
      })
      .then((response) => {
        if (!response) {
          throw new Error({ msg: 'error' });
        }
        setQrcode(response.data.qrcode);      Patricio Cruz, 3 weeks ago • Actualizacion Completa
      })
      .catch((error) => {
        console.log(error);
        setQrcode(null);
```

En el *backend* se creó un *viewset UserQrcodeView* que consulta todos los usuarios en búsqueda del código *QR* correspondiente, esto se consiguió con el serializador por modelos *UserQrcodeSerializer,* que responde a todas las consultas con el identificador del usuario y su código *QR* en formato *base64*, como se visualiza en la figura 24. Además, se creó un *script* que utiliza la librería *qrcode* de *Python*, donde se genera un identificador único para el usuario utilizando como referencia el espacio de nombres *DNS,* que utiliza el

servidor y este *UUID* es embebido en una ruta *URL, y* a su vez es convertida en el código *QR* y pasado a formato *base64*, el código del *script* se aprecia en la figura 25.

Figura 24. Código del Serializador USerQrcodeSerializer

```
    vehicles = VehicleSerializer(many = True)
    Patricio Cruz, 3 weeks ago | 1 author (Patricio Cruz)
    class Meta:
        model = User
        fields = ["id", "username", "email", "date", "first_name", "last_name", "vehicles"]
        read_only_field = ["id"]

Patricio Cruz, 3 weeks ago | 1 author (Patricio Cruz)
class UserQrcodeSerializer(serializers.ModelSerializer):      Patricio Cruz, 3 weeks ago • Actualizacion Comp
    Patricio Cruz, 3 weeks ago | 1 author (Patricio Cruz)
    class Meta:
        model = User
        fields = ["id", "qrcode"]
        read_only_field = ["id", "qrcode"]
```

Figura 25. Código del Script create_qrcode.py

```
from uuid import uuid3, NAMESPACE_DNS
from io import BytesIO
import qrcode
from base64 import b64encode


def create_qrcode(username: str = "", url: str = "") -> list:
    try:
        # Creat Unique Identifier by the username and hash MD5
        uuid = uuid3(NAMESPACE_DNS, username)
        # Create QRCODE for the user and encode to base64
        qr = qrcode.QRCode(
            version=1,
            error_correction=qrcode.constants.ERROR_CORRECT_L,
            box_size=10,
            border=4,
        )
        qr.add_data(f'{url}/check/{uuid.hex}')
        qr.make(fit=True)
        img = qr.make_image(fill_color="black", back_color="white")

        buffered = BytesIO()
        img.save(buffered)
        img_str = b64encode(buffered.getvalue()).decode("utf-8")

        return [img_str, uuid]
    except Exception:      Patricio Cruz, 3 weeks ago • Actualizacion Completa
        print(Exception.args)
        return ["", ""]
```

4.3.3.2.2. Historia de Usuario 4: Registrar Vehículos

La información detallada en la historia se puede hallar en el anexo V. En el desarrollo de esta interfaz se utilizó la librería de *React* y componentes de *MUI* para crear la página *VehicleShow.jsx*, que se encarga de visualizar varias tarjetas que representa los vehículos con su información, como se visualiza en la figura 26 desde la computadora y la figura 27 desde el teléfono.

Figura 26. Interfaz de VehicleShow.jsx, vista de escritorio

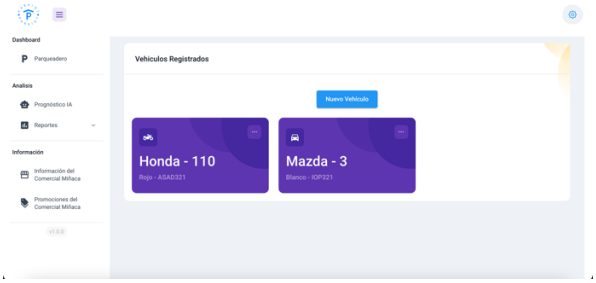
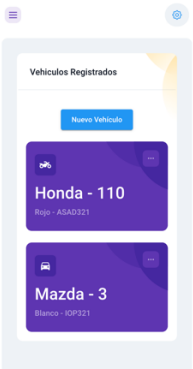


Figura 27. Interfaz de VehicleShow.jsx, vista móvil



En la misma sección del Registro de Vehículos, se creó la página *VehicleCreate.jsx,* que utiliza componentes de *MUI* y *Formik* para permitir crear un formulario e ingresar nuevos vehículos, como se muestra en la figura 28 desde la computadora y la figura 29 desde el teléfono.

Figura 28. Interfaz de VehicleCreate.jsx, vista de escritorio

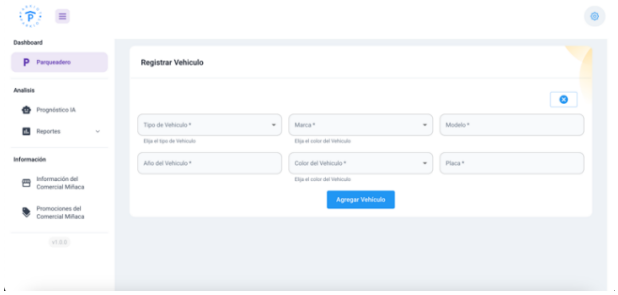
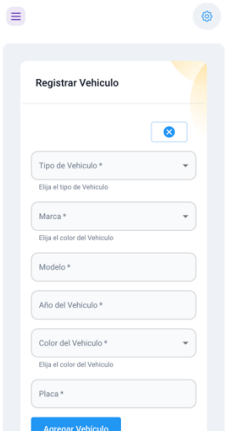


Figura 29. Interfaz de VehicleCreate.jsx, vista móvil



Para crear las páginas se utilizaron las librerías de *React* y *MUI*, primero se creó el componente *VehicleCard.jsx,* que define la información detallada de cada uno de los vehículos, esta tarjeta incluye la opción de actualizar los datos o eliminar el vehículo

vinculado al usuario, la estructura de la página se puede apreciar en el código de la figura 30, por lo tanto, se creó un componente base denominado *VehicleShow.jsx,* que muestra una tarjeta por vehículo relacionado al usuario, como se plasma en el código de la figura 31.

Figura 30. Código de VehicleCard.jsx

```
return (
  <>
    {isLoading ? (
      <SkeletonEarningCard />
    ) : (
      <CardWrapper border={false} content={false} ischecked={vehicleId === vehicle.id ? 'true' : 'false'}>
        <Box sx={{ p: 2.25 }}>
          <Grid container direction="column">
            <Grid item>
              <Grid container justifyContent="space-between">
                <Grid item>
                  <Avatar
                    variant="rounded"
                    sx={{
                      ...theme.typography.commonAvatar,
                      ...theme.typography.largeAvatar,
                      backgroundColor: theme.palette.secondary[800],
                      mt: 1
                    }}
                    onClick={handleCheckVehicle}
                  >
                    {vehicle.tipo === 'carro' ? (
                      <DirectionsCar stroke={1.5} size="1.3rem" sx={{ color: 'white' }} />
                    ) : (
                      <TwoWheeler stroke={1.5} size="1.3rem" sx={{ color: 'white' }} />
                    )}
                  </Avatar>
                </Grid>
                {!isForCheck && (
                  <Grid item>
                    <Avatar
                      variant="rounded"
```

Figura 31. Código de VehicleShow.jsx

```
//MUI
import { Grid, Box } from '@mui/material';
import { Button } from '@mui/material';

import AnimateButton from 'ui-component/extended/AnimateButton';

import configData from '../../config';
import VehicleCard from 'components/VehicleCard';
import GeneralBack from 'components/GeneralBack';

const VehicleShow = () => {
  const account = useSelector((state) => state.account);
  const { vehicles, slots } = useSelector((state) => state.vehicles);
  const dispatcher = useDispatch();
  const [disableAddButton, setDisableAddButton] = useState(false);
  const navigate = useNavigate();

  const handleAddVehicle = () => {
    if (slots < 3) {
      navigate('/vehicle/create');
    }
  };

  useEffect(() => {
    axios
      .get(`${configData.API_SERVER}/parking/vehicle/`, {
        headers: {
          Authorization: `${account.token}`
        }
      }}
      .then((response) => {
        dispatcher({
          type: SET_VEHICLES,
```

Se creó la página *VehicleCreate.jsx* con la librería *Formik* para crear los formularios de registro de vehículos, validar los datos y manejar los errores, además de la librería *Axios* para enviar los datos al *backend*, como se visualiza en la figura 32.

Figura 32. Código de VehicleCreate.jsx

```
<Formik
  initialValues={{
    tipo: '',
    brand: '',
    model: '',
    year: '',
    color: '',
    placa: ''
    // submit: null
  }}
  validationSchema={Yup.object().shape({
    model: Yup.string().max(255, 'El maximo es: 255').required('Se requiere el Modelo del Vehiculo'),
    year: Yup.number()
      .min(1500, 'El año minimo es: 1500')
      .max(anioActual, `El año maximo es: ${anioActual}`)
      .required('Se requiere el Año del Vehiculo'),
    placa: Yup.string()
      .min(6, 'Minimo de caracateres es: 6')
      .max(7, 'Maximo de caracteres es: 7')
      .required('Se requiere la Placa del Vehiculo')
  })}
  onSubmit={async (values, { setErrors, setStatus, setSubmitting }) => {
    await axios
      .post(`${configData.API_SERVER}/parking/vehicle/`, values, {
        headers: { Authorization: `${account.token}` }
      })
      .then((response) => {
        if (response.status === 201) {
          setSubmitting(true);
          setStatus({ success: true });
          navigate('/vehicle/show');
        } else {
```

En el lado del *backend* se definió el modelo *Vehicle* con todos los campos que debe poseer cada instancia del objeto que se cree, como se está utilizando el *ORM* de *Django*, automáticamente se genera la estructura de información en la base de datos correspondientes al modelo que se muestra en la figura 33.

Figura 33. Código del Modelo Vehicle

```
You, 2 hours ago | 2 authors (Patricio Cruz and others)
class Vehicle(models.Model):
    brand = models.CharField(max_length=45, verbose_name="marca")
    model = models.CharField(max_length=45, verbose_name="modelo")
    color = models.CharField(max_length=45)
    tipo = models.CharField(max_length=45, choices=TIPO_CHOICES)
    placa = models.CharField(max_length=45, unique=True)
    year = models.IntegerField(verbose_name="año del vehiculo")
    owner = models.ForeignKey(
        User, on_delete=models.CASCADE, verbose_name="dueño", related_name='vehicles', null=True, blank=True)

    Patricio Cruz, 3 weeks ago • Actualizacion Completa
```

Para llevar a cabo operaciones de *CRUD* sobre el modelo se crearon dos vistas *VehicleViewList,* que permite consultar una lista de todos los vehículos y crear nuevos vehículos como se muestra en la figura 34 y *VehicleViewDetail,* que permite actualizar y eliminar un vehículo en específico, como se visualiza en la figura 35.

Figura 34. Código de la vista VehicleViewList

```
You, 2 hours ago | 2 authors (Patricio Cruz and others)
class VehicleViewList(APIView):
    permission_classes = [IsAuthenticated,]

    def get(self, request, format=None):
        user = self.request.user
        vehicles = user.vehicles.all()
        # vehicles = Vehicle.objects.filter(owner=user)
        serilizer = VehicleSerializer(vehicles, many=True)
        return Response(serilizer.data, status=HTTP_200_OK)

    def post(self, request, format=None):
        user = self.request.user
        serializer = VehicleSerializer(data=request.data)
        if serializer.is_valid(raise_exception=True) and user.vehicles.count() < 3:
            serializer.save(owner=user)
            return Response(serializer.data, status=HTTP_201_CREATED)
        return Response({"success": False, "msg": "No se puede crear vehiculos"}, status=HTTP_400_BAD_REQUEST)
```

Figura 35. Código de la vista VehicleViewDetail

```
You, 2 hours ago | 2 authors (Patricio Cruz and others)
class VehicleViewDetail(APIView):
    permission_classes = [IsAuthenticated,]

    def get_object(self, pk: int):
        try:
            return Vehicle.objects.get(pk=pk, owner=self.request.user)
        except Vehicle.DoesNotExist:
            return Http404

    def put(self, request, pk, format=None):
        vehicle = self.get_object(pk)


    def delete(self, request, pk, format=None):
        vehicle = self.get_object(pk)
        # vehicle.delete()
        vehicle.owner = None
        vehicle.save(update_fields=['owner'])
        return Response(status=HTTP_204_NO_CONTENT)
```

Para que la información sea enviada al *frontend*, esta debe ser convertida del formato *Python Data Object* a *JSON*, para esto se creó el serializador *VehicleSerializare*, que define los campos que se enviaran en la consulta y una función personalizada para la generación de objetos de la clase *Vehicle*, esta lógica se puede apreciar en el código de la figura 36.

Figura 36. Código del serializador VehicleSerializer

```
You, 2 hours ago | 2 authors (Patricio Cruz and others)
class VehicleSerializer(serializers.ModelSerializer):
    placa = serializers.CharField(
        validators=[
        UniqueValidator(
            # queryset=Vehicle.objects.all(),
            queryset=Vehicle.objects.filter(owner = not None),
            message="El Vehiculo ya se encuentra Registrado"
        )]
    )

    Patricio Cruz, 3 weeks ago | 1 author (Patricio Cruz)
    class Meta:
        model = Vehicle
        fields = ['id', 'brand', 'model', 'color',
                  'tipo', 'placa', 'year']
        read_only_field = ["id"]

    def create(self, validate_data):
        try:
            v = Vehicle.objects.get(placa = validate_data['placa'], owner = None)
            v.owner = validate_data['owner']
            v.save(update_fields=['owner'])
            return v
        except Vehicle.DoesNotExist:
            return Vehicle.objects.create(**validate_data)
```

4.3.3.2.3. Historia de Usuario 5: Visualizar Estacionamientos

La información detallada en la historia se puede hallar en el anexo V. Para desarrollar la interfaz se utilizó la librería de *React* y componentes de *MUI*. Para presentar la información de cada estacionamiento se crearon tarjetas que representen cada espacio, la información es el número de estacionamiento, la ubicación, el color determina si está libre u ocupado y además tiene un símbolo que representa si el estacionamiento es para un automóvil, motocicleta, camioneta y furgoneta, como se visualiza en la figura 37 vista desde el computador y la figura 38 vista desde el teléfono.

Figura 37. Interfaz de ParkingShow.jsx, vista de escritorio

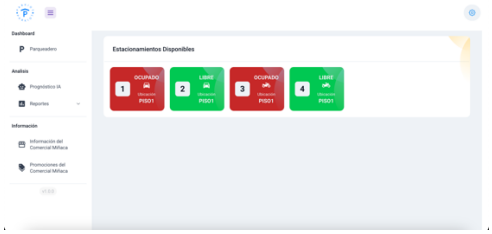


Figura 38. Interfaz de ParkingShow.jsx, vista móvil

Para la interfaz se creó el componente *ParkingCard.jsx,* con la librería de *React* y componentes de *MUI*, se creó una plantilla de tarjetas que representen cada estacionamiento como se muestra en la figura 39. Además, se utilizaron estas tarjetas para representar varias instancias de los estacionamientos en la página *ParkingShow.jsx*, dentro de esta página se utilizó el componente de *MUI Grid,* que permitió cuadrar los demás componentes en la página con un comportamiento responsivo. También, se utilizó la librería de *Axios* para traer la información de los estacionamientos desde el *backend* como se visualiza en la figura 40.

Figura 39. Código de ParkingCard.jsx

```
    height: 210,
    background: `linear-gradient(140.9deg, ${theme.palette.primary[200]} -14.02%, rgba(144, 202, 249, 0) 77.58%)`,
    borderRadius: '50%',
    top: -160,
    right: -130
  }
}));

// ============================|| DASHBOARD - TOTAL INCOME DARK CARD ||============================ //

const ParkingCard = ({ isLoading, space }) => {
  const theme = useTheme();

  return (
    <>
      {isLoading ? (
        <TotalIncomeCard />
      ) : (
        <CardWrapper border={false} content={false} estado={space.state.toString()}>
          <Box sx={{ p: 2 }}>
            <List sx={{ py: 0 }}>
              <ListItem alignItems="center" disableGutters sx={{ py: 0 }}>
                <ListItemAvatar>
                  <Avatar
                    variant="rounded"
                    sx={{
                      ...theme.typography.commonAvatar,
                      ...theme.typography.largeAvatar,
                      backgroundColor: theme.palette.grey[100],
                      color: 'ThreeDDarkShadow'
                    }}
                  >
```

Figura 40. Código de ParkingShow.jsx

```
import { useState, useEffect } from 'react';
import { useSelector } from 'react-redux';
import axios from 'axios';
import { Grid } from '@mui/material';
import ParkingCard from 'components/ParkingCard';
import configData from '../../config';
import GeneralBack from 'components/GeneralBack';

const ParkingShow = () => {
  const account = useSelector((state) => state.account);
  const [spaces, setSpaces] = useState([]);
  const [completed, setCompleted] = useState(false);

  useEffect(() => {
    axios
      .get(`${configData.API_SERVER}/parking/space/`, {
        headers: {
          Authorization: `${account.token}`
        }
      })
      .then((response) => {
        setSpaces(response.data);
        setCompleted(true);
      });
  }, []); // eslint-disable-line react-hooks/exhaustive-deps

  return (
    <>
      <GeneralBack title="Estacionamientos Disponibles">
        <Grid container spacing={2}>
          {spaces.map((data) => (
            <Grid item xs={12} md={2} key={data.id}>
```

En el *backend,* se creó el modelo *Space* que define la estructura de información de cada estacionamiento, estos campos se convirtieron en columnas de la tabla *Space* que se crea automáticamente gracias al *ORM* de *Django*, el modelo se visualiza en la figura 41. Además, se creó un serializador por modelo denominado *SpaceSerializer*, este ayuda a convertir la información de los objetos en *Python data type* a *JSON* para ser enviado al *frontend*, como se observa en la figura 42.

Figura 41. Código del modelo Space

```
class Space(models.Model):
    number = models.IntegerField(verbose_name="numero", unique=True)
    location = models.CharField(max_length=45, verbose_name="ubicacion")
    # "This state is equal to free or not free os the parking
    # "true is equivalent of free and false of not free.
    state = models.BooleanField(verbose_name="estado")
    sensor = models.CharField(
        max_length=45, verbose_name="numero sensor", unique=True)
    tipo = models.CharField(max_length=45, choices=TIPO_CHOICES)
```

Figura 42. Codigo del serializador SpaceSerializer

```
class SpaceSerializer(serializers.ModelSerializer):
    class Meta:
        model = Space
        fields = '__all__'
        read_only_field = ['number', 'location']
```

Por último, se crearon las vistas *SpaceViewList* que permite obtener una lista de todos los registros de los espacios de estacionamiento y a su vez crearlos, y la vista *SpaceViewDetail* que permite actualizar el estado y la información de un estacionamiento en específico, estas vistas se pueden apreciar en la figura 43.

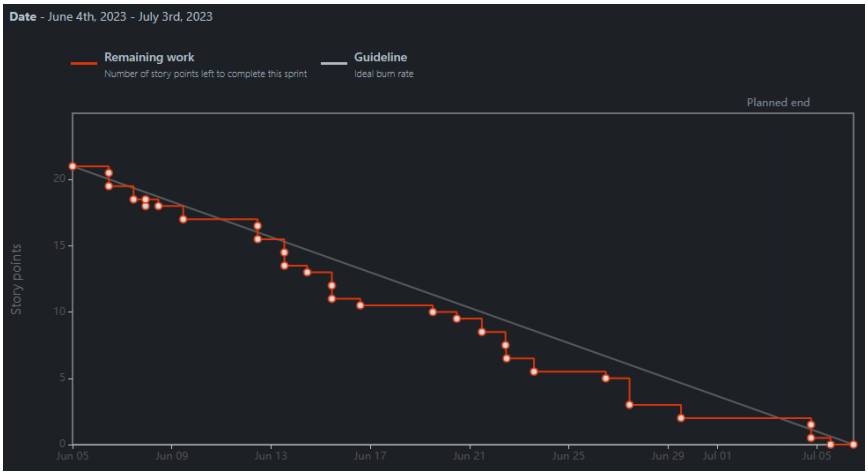
Figura 43. Código de las vistas SpaceViewList y SpaceViewDetail

```
class SpaceViewList(mixins.ListModelMixin,
                    mixins.CreateModelMixin,
                    generics.GenericAPIView):
    queryset = Space.objects.all()
    serializer_class = SpaceSerializer
    permission_classes = [IsAuthenticated,]

    def get(self, request, *args, **kwargs):
        return self.list(request, *args, **kwargs)

    def post(self, request, *args, **kwargs):
        return self.create(request, *args, **kwargs)


class SpaceViewDetail(mixins.UpdateModelMixin,
                      mixins.RetrieveModelMixin,
                      generics.GenericAPIView):
    queryset = Space.objects.all()
    serializer_class = SpaceSerializer
    permission_classes = [AllowAny,]

    def get(self, request, *args, **kwargs):
        return self.retrieve(request, *args, **kwargs)

    def put(self, request, *args, **kwargs):
        self.partial_update(request, *args, **kwargs)
        return Response({"success": True}, status=HTTP_200_OK)
```

4.3.3.2.1. Sprint 1 - Gráfico de trabajo pendiente

Para el desarrollo del *sprint* 1, se utilizó el gráfico de trabajo pendiente como una guía para conocer las tareas por cumplir de las historias de usuario. Por lo tanto, se percibió la reducción de puntos de historia faltantes por completar en el sprint, como se puede visualizar en la figura 44, el gráfico de trabajo pendiente autogenerado por la herramienta de gestión de incidencias *Jira*.

Figura 44. Gráfico de trabajo pendiente del Sprint 1 generado por Jira



4.3.3.3. Sprint 1 - Revision (*Sprint Review*)

Culminado el desarrollo del primer *sprint*, se procedió a realizar la revisión entre los integrantes de desarrollo y el gerente del local comercial, se le indicaron los avances realizados sobre el sistema de estacionamiento y se verificó el progreso del desarrollo. La reunión duró alrededor de 2 horas, donde se revisaron las historias de usuarios con sus correspondientes escenarios de pruebas y las pruebas de aceptación que se encuentran en el anexo VII.

4.3.3.3.1. Pruebas unitarias (Unit Testing)

El propósito de las pruebas unitarias, es verificar el correcto funcionamiento de la estructura y lógica de la aplicación web. De esta forma se puede verificar, que se está obteniendo el comportamiento deseado y en caso de detectar problemas, se pueden realizar correcciones al código.

Durante la etapa de desarrollo de las historias de usuarios, se ejecutaron pruebas unitarias, con la técnica de caja blanca, por lo que se centra en el funcionamiento del código y estas pruebas son automatizadas. Para llevar a cabo estas pruebas unitarias se utilizó las funciones de *UnitTest* integradas en el *framework Django*, en la figura 45 se puede visualizar la evidencia de un caso de prueba unitaria de las funcionalidades de autenticación como: *login, register* y *logout*.

Figura 45. Prueba Unitaria del Login, Register y Logout

```
Patricio Cruz, last month | 1 author (Patricio Cruz)
from django.urls import reverse
from rest_framework.test import APITestCase
from rest_framework import status


Patricio Cruz, last month | 1 author (Patricio Cruz)
class AuthenticationTest(APITestCase):
    base_url_register = reverse("api:register-list")
    base_url_login = reverse("api:login-list")
    base_url_logout = reverse("api:logout-list")
    base_url_check_session = reverse("api:check-session-list")

    data_register = {"username": "test", "password": "pass", "email": "test@parkiot.com"}

    data_login = {"password": "12345678", "email": "teast@admin.com"}

    def test_register(self):
        response = self.client.post(
            f"{self.base_url_register}", data=self.data_register
        )
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)
        response_data = response.json()
        self.assertEqual(response_data["success"], True)

    def test_login(self):
        response = self.client.post(f"{self.base_url_login}", data=self.data_login)
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        response_data = response.json()
        self.assertEqual(response_data["success"], True)

    def test_logout(self):
        # login to retrieve token
```

4.3.3.4. Sprint 1 - Retrospectiva (*Sprint Retrospective*)

Para obtener la retrospectiva del *sprint* 1, se formularon tres preguntas fundamentales con el fin de obtener una perspectiva clara, completa y concisa, como se puede apreciar en la tabla 23.

Tabla 23. Sprint 1 - Retrospectiva

Aspectos exitosos en el Sprint	***Mejoras que se podrían aplicar en el siguiente Sprint (recomendaciones para la mejora continua)***	***Errores en el Sprint***
Durante el desarrollo del *sprint* 1, se concluyó con éxito todas las tareas de ingeniería, completando así las operaciones de inicio de sesión (*login*), registro de clientes, generación de código *QR*, registro de vehículos y visualización de estacionamientos, correspondientes a cada historia de usuario. Además, se utilizó el *ORM* incorporado en *Django* para establecer conexión con la base de datos *MySQL* y almacenar la información de los modelos.	En la planificación del próximo *sprint*, se tomará en cuenta la documentación de *Django, React y Redux* como recursos de apoyo para abordar los problemas. Además, se definirán funciones individuales que ayudan a la detección de los problemas y corrección de los mismos de manera ágil.	Dentro del proceso de desarrollo del s*print* 1, se tuvo dificultades con el registro de vehículos. El error se detectó al momento de crear un nuevo vehículo, este proceso no vinculaba el vehículo con el usuario que tenía la sesión activa en el navegador *web*.

4.3.4. Sprint 2

4.3.4.1. Sprint 2 - Planificación (*Sprint Planning)*

En lo que respecta al *sprint planning* del *sprint* 2, se realizó un análisis de las historias de usuario 6 y 7, además de la incorporación de la historia técnica 1. El resultado global fueron 23 puntos de estimación, que son necesarios para completar el sprint, de igual forma se elaboró el *sprint backlog* que se visualiza en la tabla 24.

Tabla 24. Sprint Backlog 2

Sprint Backlog							
Objetivo: Elaborar el PMV con las funciones de detección vehicular, pronóstico de clientes y reportes de E/S vehicular							
Sprint	***Historia***	***Est***	***Categoría***	***Tarea***	***Est***	***Responsable***	***Estado***
2	HT1 - Interoperabilidad. Rendimiento - Detección Vehicular	8	Diseño	Diseñar las conexiones entre el sensor de proximidad y el microcontrolador. (*ESP8266, HC-SR04*)	1	David Carrascal	Completo
			Diseño	Diseñar la fuente de energía del sensor y el microcontrolador.	0,5	David Carrascal	Completo